

零知识证明论文阅读与研究设想

第一次组会汇报

徐诗霖

电子科技大学

2026 年 7 月



本次汇报想回答的问题

- 零知识证明到底解决了什么信任问题？
- 近期阅读的论文分别推进了哪一类能力？
- 从论文中学到哪些可迁移的研究方法？
- 是否能提出一个足够小、可实验验证的新方向？

汇报主线

从“证明一个结果正确”走向“证明一个 AI 系统按承诺、权限与策略执行”。

基本形式

证明者向验证者证明存在秘密见证 w ，使公开关系 $R(x, w) = 1$ 成立，但不泄露 w 本身。

$$\exists w : R(x, w) = 1$$

- x : 公开陈述，如承诺、输出、模型哈希、策略版本。
- w : 秘密见证，如输入、模型参数、中间计算 trace。
- π : 证明，验证者只需检查 $\text{Verify}(x, \pi) = 1$ 。

三条性质与能力边界

核心性质

- **完备性**：诚实执行应当通过验证。
- **可靠性**：错误声明难以伪造证明。
- **零知识性**：证明过程不泄露秘密。

需要保持清醒

- 不证明输入数据真实。
- 不证明模型公平、无偏或无后门。
- 不证明需求和规则本身合理。
- 证明的是“按给定规则正确执行”。

技术谱系：我目前的理解

技术路线	典型思想	优势	适合场景
Sigma/Schnorr	承诺--挑战--响应	简洁直观	身份认证
Groth16	电路特化 SNARK	证明极短	链上验证
PLONK	通用约束系统	工程生态好	通用应用
STARK	哈希与透明设置	无可信设置	大规模计算
GKR/Sumcheck	分层计算校验	适合矩阵/ML	ZKML、训练证明

- 我的切入点：不急着追所有协议细节，先理解“计算如何被表示、压缩和验证”。

已读论文地图

论文	关注问题	对我的启发
DIZK, USENIX Security 2018	分布式生成 zkSNARK	证明系统也是系统工程，瓶颈常在内存与并行化
zkCNN, CCS 2021	CNN 推理和准确率证明	卷积、ReLU、池化需要专门设计证明协议
Mystique, USENIX Security 2021	ZKML 中的类型转换与矩阵乘	真实 ML 不是单一算术电路，转换成本很关键
Kaizen, CCS 2024	深度网络训练证明	递归组合让多轮训练的证明保持简短
FAIRZK, arXiv 2025	公平性证明	可绕开逐样本推理，用模型参数与聚合统计证明性质
SAGA / CHaT, 2025–2026	Agent 安全治理与对抗	ZKP 可与身份、策略、审计结合，服务可信 Agent

DIZK: 把证明生成从单机推向集群

- 问题：传统 zkSNARK prover 是单机单体进程，内存随电路规模迅速膨胀。
- 方法：将证明生成拆分到计算集群，处理多项式、FFT、约束计算等并行任务。
- 结果：可扩展到十亿级逻辑门，论文报告比先前系统支持更大规模计算。
- 我的理解：ZKP 落地不只是协议正确，还要解决数据划分、通信、容错与资源调度。

Howard Wu et al., DIZK, USENIX Security 2018

zkCNN 与 Mystique: ZKML 的核心难点

zkCNN

- 面向 CNN 预测和准确率证明。
- 针对卷积、ReLU、MaxPool 设计交互式证明。
- 启发：利用 ML 结构，而不是直接套通用电路。

Mystique

- 关注算术/布尔、定点/浮点、承诺/认证值转换。
- 优化矩阵乘证明，面向大规模神经网络推理。
- 启发：转换层是系统性能的隐藏主战场。

Kaizen 与 FAIRZK：从推理正确到训练/性质正确

- **Kaizen**：证明模型确实按指定训练算法和数据承诺完成训练；递归组合使证明大小和验证时间不随迭代数增长。
- **FAIRZK**：证明模型满足公平性约束；关键不是逐样本证明每次推理，而是用模型参数与聚合信息构造公平性上界。
- 共同趋势：ZKP 正在从“证明一次计算输出”扩展到“证明模型来源、训练过程和高层性质”。

对我的启发

好的 ZK 应用往往不是把原任务硬塞进电路，而是重新寻找可证明、可压缩、可验证的等价表述。

Agent 安全论文给 ZKP 的新入口

- **SAGA**: 强调 Agent 身份、注册、访问控制 token、跨 Agent 通信策略。
- **Cloak, Honey, Trap**: 展示 LLM Agent 会被诱导、误导、循环和反制，安全边界非常脆弱。
- 共同问题: Agent 行为越来越自治，但用户和第三方很难确认它是否按权限、策略、版本执行。

连接点

Agent 治理需要“可验证执行记录”；ZKP 可以让验证者相信合规性，而不暴露完整提示词、私有工具输入或业务数据。

阶段性心得：我学到的四件事

- ① **先定义可验证关系**：论文的核心常常藏在 $R(x, w)$ 怎么选。
- ② **避免天真电路化**：直接证明完整系统通常太慢，要找结构化捷径。
- ③ **区分证明目标**：推理正确、训练正确、公平性、权限合规是不同命题。
- ④ **工程指标同样重要**：证明时间、内存、证明大小、验证时间决定能否落地。

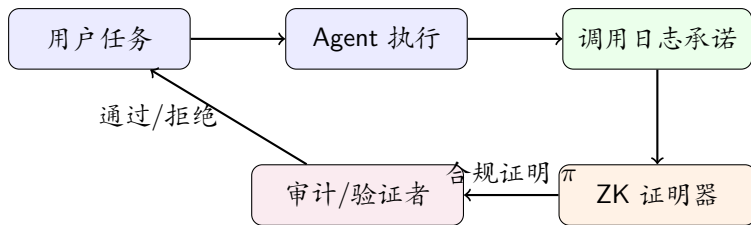
- 从应用痛点出发：谁不信任谁？秘密是什么？公开可验证的承诺是什么？
- 把系统拆成公开输入、秘密见证、中间 trace、验证关系四部分。
- 优先寻找“证明高层性质”的路径，而不是逐步证明每次底层计算。
- 做实验时先建最小原型：小模型、小策略、小数据集，跑通证明链路后再优化。

一句话

为 LLM Agent 生成“权限合规证明”：证明它在一次任务中只调用了被授权工具、使用了指定策略版本、没有越权访问敏感资源，同时不公开完整上下文。

- 适用场景：企业内部 Agent、科研助理 Agent、自动化运维 Agent。
- 公开输入：Agent 身份、策略哈希、工具白名单哈希、任务输出摘要。
- 秘密见证：完整调用日志、用户私有输入、工具返回内容、内部推理状态摘要。

ZK-AgentGuard: 系统草图



- 先不证明大模型推理本身，只证明“工具调用序列满足策略自动机”。
- 将复杂 AI 行为约束为有限状态机或规则 DSL，更容易电路化。

最小可行实验设计

模块	第一版做法
Agent 任务	日程安排、文件检索、报销提交三类简单流程
策略语言	工具白名单 + 调用次数 + 参数范围 + 顺序约束
证明命题	日志承诺中的每一步均满足策略自动机转移
原型工具	Circom/Noir/halo2 选一种先实现小电路
评价指标	证明时间、验证时间、电路约束数、日志长度扩展性

预期成果

一个可演示 demo: 验证者看到任务输出和证明, 但看不到完整用户隐私和工具返回内容。

可能的难点与应对

- **语义鸿沟**：自然语言策略难以直接证明。
应对：先设计结构化策略 DSL，再由自然语言映射到 DSL。
- **日志可信性**：Agent 可能不记录真实调用。
应对：工具代理层强制记录并签名，Agent 只能通过代理调用工具。
- **性能压力**：调用日志很长时约束数增长。
应对：用 Merkle 承诺、分段证明、递归聚合降低验证成本。
- **威胁模型**：证明合规不等于证明输出正确。
应对：先明确只证明权限与流程合规，后续再叠加输出性质证明。

下一阶段计划

- ① 补齐基础：Schnorr、Sigma 协议、Sumcheck/GKR、承诺方案。
- ② 深读两条线：ZKML 推理证明与 Agent 治理安全。
- ③ 完成 ZK-AgentGuard 的形式化定义：公开输入、见证、关系、威胁模型。
- ④ 做最小原型：先证明 10-100 步工具调用日志满足策略。
- ⑤ 输出阶段成果：技术调研文档 + demo + 性能表格。

谢谢老师和同学们

欢迎批评指正